

TÀI LIỆU ĐẶC TẢ YÊU CẦU KỸ THUẬT & DESIGN SPECIFICATION

Milestone 2: Hoàn thiện Pipeline Thị giác máy tính (ArUco Detection) — Component Level
Phần cứng nhúng: Jetson Orin Nano | Cảm biến: Siyi A8 IP Camera (RTSP) | Framework: ROS 2 Humble

1. Tổng quan hệ thống & Mục tiêu công nghệ

Tài liệu này mô tả chi tiết ở cấp độ thành phần (Component Level) cho luồng xử lý thị giác máy tính nhận diện mục tiêu hạ cánh ArUco Marker phục vụ cho Milestone 2 thuộc hệ thống máy bay không người lái (UAV). Trọng tâm của cột mốc này là tối ưu hóa tài nguyên phần cứng nhúng mạnh mẽ của NVIDIA Jetson Orin Nano, giải phóng tối đa tài nguyên CPU để dành cho các tác vụ định vị toàn cục và lập kế hoạch quỹ đạo bay, đồng thời đảm bảo độ trễ và độ chính xác của dữ liệu ngấm bắn trước khi chuyển giao sang mạch điều khiển bay (Flight Controller — PX4).

Hệ thống sử dụng luồng dữ liệu hình ảnh kỹ thuật số độ phân giải cao thông qua giao thức RTSP từ Camera IP Siyi A8. Để đáp ứng cấu trúc khắt khe của mô hình chữ V (V-Model), toàn bộ các giao diện đầu vào, đầu ra, điều kiện biên và ma trận kiểm chứng nghiệm thu đều được định nghĩa rõ ràng về mặt cấu trúc dữ liệu vật lý và logic toán học.

2. Đặc tả bản tin dữ liệu mở rộng (Data Message Specification)

Để giải quyết triệt để bài toán biến đổi hệ quy chiếu động và tích hợp giải thuật 'Tag lỏng Tag', bản tin nguyên bản đã được cấu trúc lại hoàn chỉnh trong không gian 6 chiều tự do (6DoF Pose) kèm nhãn thời gian đồng bộ.

2.1. Cấu trúc bản tin nội bộ: dib_msgs/msg/LandingTarget6D.msg

Vị trí tệp trong workspace: /dib_msgs/msg/LandingTarget6D.msg — cần tạo mới (có thể trao đổi để điều chỉnh cấu trúc msg cho phù hợp thực tiễn)

Thông tin landing target mở rộng định vị 6D không gian phục vụ hạ cánh chính xác
std_msgs/Header header

Tọa độ vị trí 3D (Position) - Đơn vị: Mét (m)

float64 x

float64 y

float64 z

Tọa độ hướng 3D (Orientation Euler Angles) - Đơn vị: Radian (rad)

float64 roll

float64 pitch

float64 yaw

Trạng thái bám bắt mục tiêu (Tracking State)

0 = LOST (Không tìm thấy mục tiêu)

1 = SEARCHING (Đã từng thấy, đang chạy thuật toán dự đoán/tìm kiếm lại)

2 = TRACKING (Đang bám bắt trực tiếp thời gian thực ổn định)

uint8 state

Định danh ArUco Tag phục vụ giải thuật lỏng ghép cự ly tiếp cận

int32 tag_id

2.2. Bản tin ROS 2 chuẩn tham chiếu

Tham chiếu: https://github.com/ros2/common_interfaces/tree/rolling/sensor_msgs/msg

3. Danh sách yêu cầu kỹ thuật chi tiết (Component Requirements)

Ghi chú cấu trúc: Module Precision Landing là thành phần con thuộc Companion PC. Các yêu cầu được tổ chức theo hai tầng:

- Yêu cầu bậc cao (High-level): Không ràng buộc thiết kế — định nghĩa cái gì cần đạt, áp dụng chung cho toàn luồng (video, điều khiển, đường truyền, tài nguyên).
- Yêu cầu đặc thù (Module-specific): Chi tiết hơn cho riêng module Precision Landing, vẫn không chỉ định giải pháp triển khai cụ thể.

3.1. Yêu cầu bậc cao — Module Precision Landing (thuộc Companion PC)

REQ ID	Tên yêu cầu	Mô tả (What — không ràng buộc thiết kế)	Tham chiếu	Ghi chú
REQ_PL_VI_DEO_001	Thu nhận luồng hình ảnh liên tục	Hệ thống phải thu nhận được luồng hình ảnh từ camera onboard và cung cấp dữ liệu hình ảnh thô cho các module xử lý hạ nguồn với độ trễ đầu cuối (end-to-end latency) không vượt quá 100ms.	Task 2.0	
REQ_PL_CT_RL_001	Cung cấp dữ liệu dẫn đường cho bộ điều khiển bay	Hệ thống phải cung cấp tín hiệu vị trí mục tiêu hạ cánh liên tục với tần số tối thiểu 20Hz đến Flight Controller.	Task 2.1	
REQ_PL_LI_NK_001	Đảm bảo đường truyền nội bộ không bị gián đoạn	Giao tiếp giữa Companion PC và Flight Controller phải đảm bảo không mất gói tin trong điều kiện hoạt động bình thường.	REQ_UA_V_TALA_0007	Không cần test trong module detect
REQ_PL_RE_S_001	Không chiếm dụng tài nguyên (CPU) vượt ngưỡng an toàn	Toàn bộ tài nguyên tính toán mà module Precision Landing sử dụng phải nằm trong ngưỡng an toàn đã phân bổ, không ảnh hưởng đến các tác vụ định vị toàn cục và lập kế hoạch quỹ đạo bay đang chạy song song.	Task 2.1	

3.2. Yêu cầu đặc thù — Module Precision Landing

REQ ID	Tên yêu cầu	Mô tả (What)	Tiêu chí nghiệm thu	Tham chiếu	Ghi chú
REQ_PL_DET_001	Nhận diện mục tiêu hạ cánh bằng fiducial marker (Aruco Tag)	Hệ thống phải nhận diện được marker hạ cánh xuất hiện trong trường nhìn camera và xác định được vị trí 6DoF (x, y, z, roll, pitch, yaw) của marker đó trong hệ tọa độ camera.	Tỉ lệ nhận diện thành công > 95% trong điều kiện ánh sáng phòng tiêu chuẩn. Tần số >20Hz, tối ưu là 30Hz	Task 2.2, TEST 2.1	
REQ_PL_DET_002	Khả năng nhận diện ổn định theo cự ly	Hệ thống phải nhận diện được mục tiêu hạ cánh tại các cự ly từ 0.5m đến 10m mà không cần thay đổi cấu hình.	Nhận diện thành công tại các mốc kiểm thử: 1m, 3m, 5m, 10m.	TEST 2.1	Test hardware

REQ_PL_NEST_001	Chuyển đổi mục tiêu theo cự ly tiếp cận (Nested Tag)	Khi có nhiều marker đồng thời trong trường nhìn, hệ thống phải tự động lựa chọn mục tiêu hạ cánh phù hợp nhất — ưu tiên marker cho phép định vị chính xác cao hơn ở cự ly gần.	Thời gian chuyển đổi mục tiêu < 0.5s; không có gián đoạn tín hiệu dẫn đường vượt quá ngưỡng này.	Task 2.4, TEST 2.2	Có thể không cần chuyển đổi, cần Đức Anh cf
REQ_PL_COORD_001	Biểu diễn tọa độ mục tiêu trong hệ quy chiếu thân máy bay	Dữ liệu vị trí mục tiêu cung cấp cho Flight Controller phải được biểu diễn trong hệ tọa độ thân máy bay (body frame) (ENU), không phải hệ tọa độ camera.	Sai số tính hình học sau chuyển đổi < 1cm (so với hệ tọa độ trước chuyển đổi) tại điều kiện kiểm thử chuẩn.	Task 2.1, TEST 2.0	
REQ_PL_COORD_002	Đảm bảo nhất quán dấu vật lý tọa độ đầu ra	Chiều dương/âm của các trục tọa độ đầu ra phải phản ánh đúng hướng dịch chuyển vật lý thực tế của mục tiêu so với UAV.	Di chuyển mục tiêu theo 4 hướng vật lý — dấu tọa độ đầu ra phản ánh đúng 100% trong cả 4 trường hợp. Trái phải tiến lùi.	TEST 2.0	
REQ_PL_OUT_001	Định dạng dữ liệu đầu ra tương thích chuẩn giao tiếp Flight Controller	Dữ liệu vị trí mục tiêu đầu ra phải tương thích với chuẩn giao tiếp của Flight Controller (PX4) — không yêu cầu chuyển đổi thêm ở phía nhận.	Dữ liệu được Flight Controller nhận và xử lý thành công mà không cần bước chuyển đổi bổ sung.	Task 2.1	Đáp ứng REQ_PL_COORD_001 là đủ

4. Design Description — Module Precision Landing (dưới đây là gợi ý, có thể làm theo kiến trúc khác chỉ cần đảm bảo hiệu quả)

Mục đích: Mô tả cách hệ thống đáp ứng từng yêu cầu đã đặt ra ở Phần 3 (REQ). Tài liệu này ràng buộc thiết kế cụ thể cho người thực thi — trả lời câu hỏi "How".

Mỗi mục thiết kế được gắn nhãn REQ tương ứng để đảm bảo traceability theo V-Model.

4.1. Kiến trúc tổng thể module

...

Viết ngay từ đầu nếu đã có hình dung kiến trúc tổng
Hoặc hoàn thiện cuối cùng đóng gói tài liệu khi đã giải quyết hết các REQ

4.2. Video Ingest Node

→ **Đáp ứng: REQ_PL_VIDEO_001**

Lưu ý: Luồng video này sử dụng lại pipeline đang được tối ưu sẵn từ team anh Phước, không tự triển khai lại.

4.3. ArUco Detection Node

→ **Đáp ứng: REQ_PL_DET_001, REQ_PL_DET_002, REQ_PL_RES_001**

Đức Anh đã có mô phỏng phần này, có thể sử dụng lại nguồn

Lộ trình thử nghiệm ArUco Dictionary (dự kiến, có thể đẩy sang giai đoạn 2 luôn):

Giai đoạn	Dictionary	Lý do
Giai đoạn 1 — Test ban đầu	DICT_4X4_50	Nhỏ gọn, detect nhanh, dễ in, kiểm thử thuật toán
Giai đoạn 2 — Production	DICT_ARUCO_MIP_36h12 Hoặc Apriltag_36h11	Hamming d=12, chống nhiễu cao nhất, mục tiêu chính
Fallback	DICT_6X6_250	Nếu MIP_36h12 không đạt detection rate yêu cầu

- Output: mảng `dib_msgs/LandingTarget6D[]`, mỗi phần tử là một tag nhận diện được.
- `header.frame_id` được gán cứng = "camera_link".
- Tần số xử lý mục tiêu: 30 FPS ổn định; CPU tổng thể toàn luồng < 30%.

4.4. Nested Tag Logic Node (Kiến nghị)

→ Đáp ứng: REQ_PL_NEST_001

Node riêng nhận mảng tag đầu vào, áp dụng logic ưu tiên dựa trên `tag_id` và khoảng cách `z`.

Quy tắc phân loại tag:

- Outer Tag (ID lớn, kích thước vật lý lớn ~30cm): dùng khi UAV ở xa ($z >$ ngưỡng chuyển đổi).
- Inner Tag (ID nhỏ, kích thước vật lý nhỏ ~4cm): ưu tiên khi UAV ở gần ($z \leq$ ngưỡng chuyển đổi) và cả hai tag cùng xuất hiện trong trường nhìn.

Cơ chế chuyển đổi — Hysteresis Switch (có thể không cần chuyển):

- Áp dụng thời gian trễ 0.5s để tránh dao động liên tục khi UAV ở vùng biên ngưỡng.
- State machine gồm 3 trạng thái: OUTER_TRACKING → SWITCHING → INNER_TRACKING.

Cách đặt tag (Cách 1 — khuyến nghị theo Phụ lục 5):

- Inner Tag nằm ở tâm Outer Tag. Kích thước: Outer ~30cm, Middle ~10cm, Inner ~4cm.

Tham chiếu:

https://github.com/dimianx/uav_landing_sim/blob/main/uav_landing_sim/models/husky/meshes/marker.png

4.5. Coordinate Transform Node

→ Đáp ứng: REQ_PL_COORD_001, REQ_PL_COORD_002

Sử dụng thư viện `tf2_ros` để tra cứu ma trận biến đổi tại thời điểm `header.stamp` của bản tin đầu vào, đảm bảo tính đồng bộ thời gian.

Phép biến đổi:

$$P_{body} = R_{cam_to_body} \times P_{cam} + T_{cam_to_body}$$

- `R_cam_to_body`: Ma trận xoay 3×3, xác định theo vị trí lắp đặt camera thực tế trên khung UAV. Publish lên TF Tree dưới dạng static transform.
- `T_cam_to_body`: Vector tịnh tiến từ vị trí camera đến tâm trọng lực UAV — đo đạc cơ học thực tế.

- Output frame_id: "base_link" (ENU) MAVROS.

Kiểm tra dấu vật lý — bắt buộc trước khi tích hợp:

Di chuyển vật lý	Giá trị tọa độ kỳ vọng (body frame FRD)
Mục tiêu sang TRÁI	x âm (body frame FRD)
Mục tiêu sang PHẢI	x dương
Mục tiêu ra TRƯỚC	y dương
Mục tiêu ra SAU	y âm

4.6. MAVLink Bridge Node

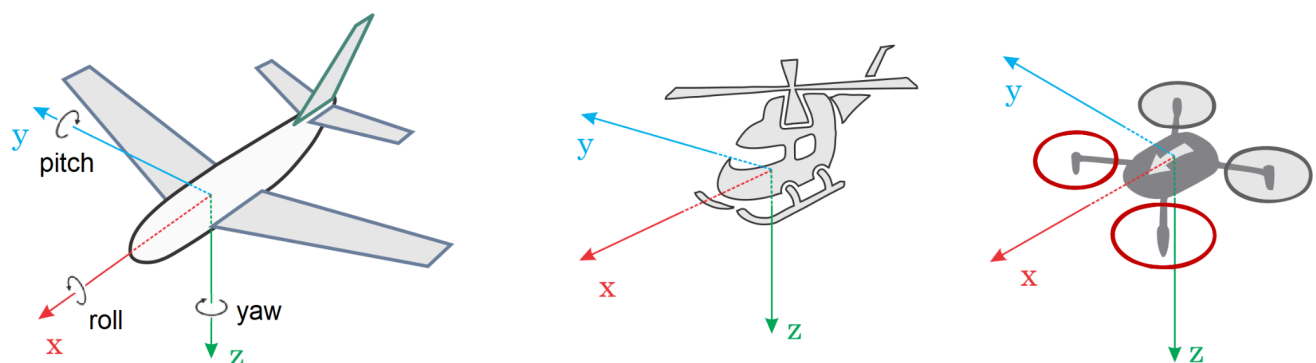
→ **Đáp ứng:** REQ_PL_OUT_001, REQ_PL_CTRL_001, REQ_PL_LINK_001

Node subscribe topic body-frame, đóng gói thành message LANDING_TARGET theo chuẩn MAVLink và publish qua MAVROS tới PX4.

- Giao tiếp vật lý: UART hoặc Ethernet nội bộ giữa Jetson Orin Nano và Pixhawk.
- Tần số publish: 30Hz (tối thiểu 20Hz để đáp ứng REQ_PL_CTRL_001).
- Nếu mất tín hiệu detection (state = LOST): dừng publish, không gửi tọa độ rác về PX4.

5. Cơ sở toán học chuyển đổi hệ tọa độ 6D không gian

Để đảm bảo tính nhất quán động học vật lý, tọa độ mục tiêu từ hệ trục Camera cần được ánh xạ chính xác về hệ trục vật lý UAV (là **ENU** - MAVROS tự động chuyển đổi ENU ↔ NED khi truyền message. Viết code phía ROS 2 thì **dùng ENU**, không cần quan tâm đến NED của PX4, <https://github.com/mavlink/mavros/blob/ros2/mavros/README.md>). Phép xoay không gian 3 chiều được tính toán tổng hợp từ 3 góc Euler (roll ϕ , pitch θ , yaw ψ) để tạo thành ma trận xoay trục giao R.



Hệ trục NED

$$P_UAV = R_cam_to_UAV \times P_cam + T_cam_to_UAV$$

Trong đó:

- $P_{cam} = [x_c, y_c, z_c]^T$ — vector vị trí ArUco Tag trong hệ tọa độ quang học camera.
- $R_{cam_to_UAV}$ — ma trận xoay 3×3 biểu diễn hướng của Camera Mount so với trục thân máy bay $base_link$.
- $T_{cam_to_UAV}$ — vector tịnh tiến $[X, Y, Z]^T$ từ vị trí lắp đặt camera đến tâm trọng lực UAV.

Nguyên tắc kiểm tra dấu vật lý: Khi dịch chuyển camera thực tế sang hướng TRÁI vật lý, giá trị tọa độ mục tiêu trả về phải dịch chuyển tương ứng sang hướng PHẢI vật lý và ngược lại. Sai số tính hình học không được vượt quá ngưỡng cho phép ($< 1cm$).

6. Kịch bản kiểm chứng nghiệm thu kỹ thuật

6.1. TEST 2.0 — Unit Test kiểm chứng ma trận chuyển đổi tọa độ

Phương pháp tiến hành:

Chạy script/notebook offline sử dụng tệp dữ liệu ảnh mẫu và video ghi nhận thực tế từ Task 2.1. Đầu vào là mục tiêu cố định đã đo đạc kích thước cơ học chính xác từ trước.

Tiêu chí Đạt (PASS):

- Toàn bộ dữ liệu vị trí đầu ra X/Y/Z sai số so với thực tế $< 1cm$ so với trước khi chuyển đổi.
- Thử nghiệm dịch chuyển camera theo 4 hướng vật lý — dấu hệ tọa độ phản ánh hoàn toàn chính xác.

6.2. TEST 2.1 — Bench Test hiệu năng trên máy tính nhúng

Phương pháp tiến hành:

Gắn cố định Camera IP Siyi A8 trên Rig kiểm thử kết nối với Jetson Orin Nano qua cổng mạng Ethernet. Đặt hệ thống Tag lồng ghép tại các mốc khoảng cách cố định (1m, 3m, 5m, 10m).

Tiêu chí Đạt (PASS):

- Framerate ổn định ≥ 20 FPS, lý tưởng đạt 30 FPS.
- Mức chiếm dụng CPU tổng thể của toàn bộ luồng thị giác không vượt quá 35%(? Có thể tốt hơn).
- Tỷ lệ phát hiện Tag thành công (Detection Rate) $> 95\%$ trên toàn bộ thời gian trong điều kiện ánh sáng phòng tiêu chuẩn.

6.3. TEST 2.2 — Dynamic Bench Test giải thuật Tag lồng Tag

Phương pháp tiến hành:

Di chuyển Rig camera tịnh tiến từ khoảng cách xa (10m) tiến lại gần sát bề mặt Tag lồng Tag (0.5m) để mô phỏng quá trình hạ cánh của drone. Theo dõi log dữ liệu dải bản tin đầu ra.

Tiêu chí Đạt (PASS):

- Thuật toán chuyển đổi mục tiêu theo dõi mượt mà giữa Outer Tag và Inner Tag.
- Tổng thời gian mất tín hiệu ngắt bắt hoặc nhiễu xung động tọa độ trong quá trình chuyển trạng thái $< 0.5s$.

6.4. Checklist nghiệm thu

- ☐ TEST 2.0: Script unit test ma trận chuyển đổi — PASS với sai số < 1cm
- ☐ TEST 2.1: Benchmark profiling — %CPU < 35%, FPS ≥ 20 (mục tiêu 30), Det. Rate > 95%
- ☐ TEST 2.2: Dynamic test Tag lồng Tag — chuyển outer → inner không mất > 0.5s (có thể lấy kết quả test để hiệu chỉnh số này cho phù hợp)
- ☐ Bảng Profiling đầy đủ: %CPU, %GPU, FPS, khoảng cách xa/gần theo từng cấu hình
- ☐ Thiết kế Tag lồng Tag (kích thước, ID, sơ đồ) — lưu tài liệu
- ☐ File ảnh tag đã in (outer 30cm, middle 10cm, inner 4cm) — kiểm tra chất lượng in
- ☐ File calibration camera
- ☐ Code node xử lý ảnh
- ☐ Báo cáo TEST 2.0, 2.1, 2.2 lưu tài liệu
- ☐ Lưu tài liệu Design kỹ thuật

PHỤ LỤC

PL1. Calibration Camera

Tài liệu hướng dẫn:

https://industrial-training-master.readthedocs.io/en/humble/_source/session9/Cameras-and-Calibration.html

PL2. Lựa chọn ArUco Dictionary

Công cụ tạo marker để in: <https://chev.me/arucogen/>

Dựa trên yêu cầu chống nhiễu và số lượng tag cần dùng (< 30 tag), bảng so sánh khuyến nghị:

Dictionary	Hamming dist.	Marker tối đa	Nhận xét	Khuyến nghị
DICT_4X4_50	d=12	50	Nhỏ gọn, detect nhanh, dễ in để kiểm thử thuật toán ban đầu	✗ Chống nhiễu thấp
DICT_APRILTAG_36h11	d=11	587	Detect tốt ở góc nghiêng và điều kiện sáng yếu	✓ OK
DICT_ARUCO_MIP_36h12	d=12	250	Chống nhiễu tốt hơn 36h11 một bậc — mục tiêu production	✓ Mục tiêu chính
DICT_6X6_250	d=12	250	Tương đương MIP 36h12 về độ bền nhiễu	✓ Thay thế tốt
DICT_7X7_1000	d=14	1000	Chống nhiễu cao nhất, cần in marker kích thước lớn hơn	⚠️ Cần nhắc

PL3. Package ROS 2 ArUco gợi ý

Package: `ros2_aruco` (JMU-ROBOTICS-VIVA) — hỗ trợ ROS 2 Humble, dùng OpenCV, hỗ trợ đa dictionary.

Link: https://github.com/JMU-ROBOTICS-VIVA/ros2_aruco/tree/humble

Lưu ý: Dictionary MIP_36h12 chưa được mô tả trong README — cần kiểm thử thực tế.

PL4. Template bảng báo cáo Profiling

Config	Resolution	Tag size	Distance	%CPU	%GPU	FPS	Det. Rate	Ghi chú
A1	640×480	20cm	5m	—	—	—	—	
A2	640×480	20cm	8m	—	—	—	—	
A3	640×480	20cm	10m	—	—	—	—	
B1	1280×720	20cm	5m	—	—	—	—	
B2	1280×720	20cm	8m	—	—	—	—	
C1	640×480	10cm	1m	—	—	—	—	
C2	640×480	10cm	2m	—	—	—	—	

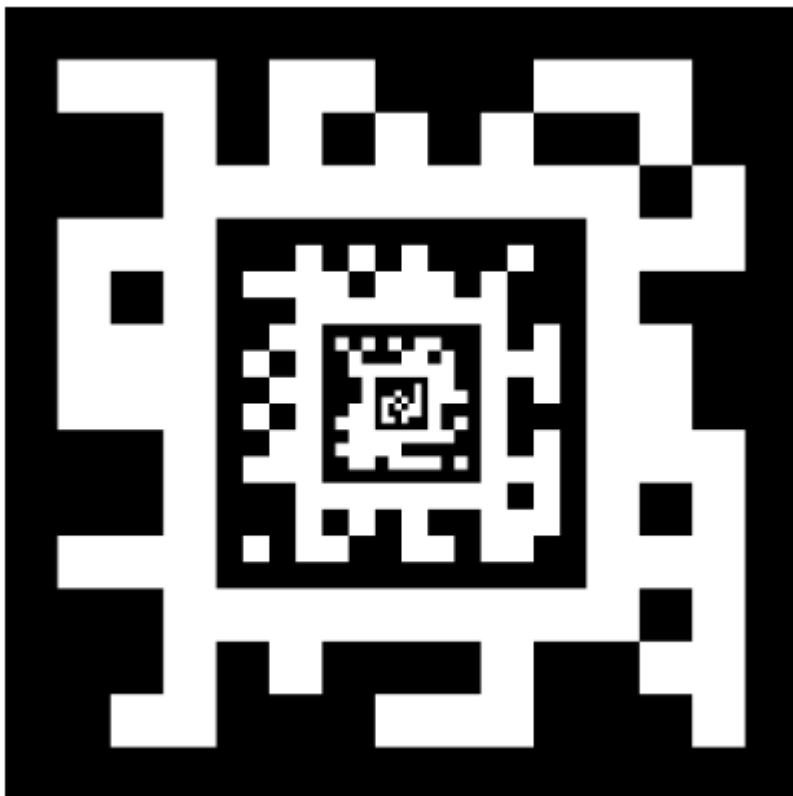
E1	640×480	4cm	0.3m	—	—	—	—	
----	---------	-----	------	---	---	---	---	--

PL5. Các cách đặt Tag lồng Tag

Cách 1 — Khuyến nghị:

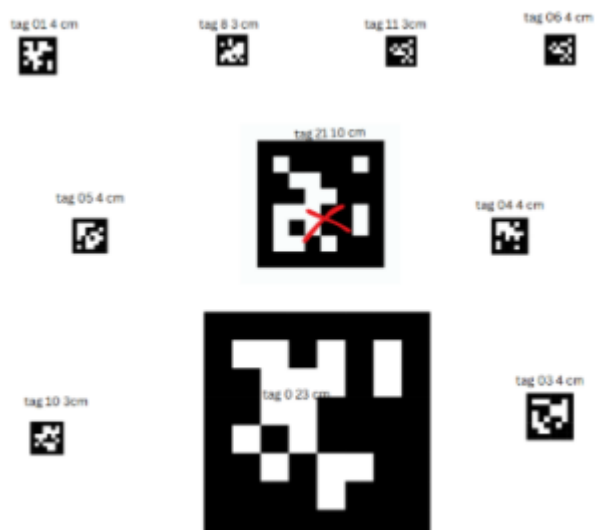
Tag nhỏ (Inner) nằm ở tâm tag lớn (Outer). Tham chiếu:

https://github.com/dimianx/uav_landing_sim/blob/main/uav_landing_sim/models/husky/meshes/marker.png



Cách 2:

Các tag nằm rải rác; vị trí tâm các tag là điểm hạ cánh.



Nhược điểm: mất thêm thời gian tính toán; khi UAV hạ gần có thể mất tag trong trường nhìn → sai lệch vị trí hạ cánh → KHÔNG khuyến nghị.

<http://10.20.14.2:8090/x/ixmXAO>

Cả hai cách đều phải xử lý dữ liệu bằng bộ lọc thông thấp — thông cao.

PL6. Tài liệu tham khảo

- GitHub UAV Landing Sim: https://github.com/dimianx/uav_landing_sim
- ROS 2 ArUco package: https://github.com/JMU-ROBOTICS-VIVA/ros2_aruco
- SIYI SDK Python: https://github.com/mzahana/siyi_sdk
- SIYI A8 Mini User Manual v1.8:
https://siyi.biz/siyi_file/A8%20mini/A8%20mini%20User%20Manual%20v1.8.pdf